



Guía: uso de la clase Files en Java



`Files` es la clase de utilidades de `java.nio.file` para trabajar con archivos y directorios. Sus métodos son estáticos y operan normalmente sobre objetos `Path`.

1. El modelo mental: Path indica dónde; Files realiza la operación

La API moderna de Java separa dos responsabilidades:

- `Path` representa una ruta en el sistema de archivos.
- `Files` permite leer, escribir, copiar, mover, borrar, consultar o recorrer esa ruta.

```
import java.nio.file.Files;
import java.nio.file.Path;

Path archivo = Path.of("datos", "clientes.csv");

if (Files.exists(archivo)) {
    System.out.println("El archivo existe");
}
```

`Path.of(...)` no crea nada en el disco: solo construye un objeto que representa una ruta. La operación real comienza al invocar un método de `Files`.



La mayoría de las operaciones de `Files` pueden lanzar `IOException`. Es una excepción comprobada: debe capturarse con `try-catch` o declararse mediante `throws`.

2. Rutas relativas y absolutas

```
Path relativa = Path.of("datos", "clientes.csv");
Path absoluta = Path.of("C:", "aplicacion", "datos", "clientes.csv");

System.out.println(relativa.toAbsolutePath());
System.out.println(relativa.normalize());
```

Una ruta relativa se interpreta desde el directorio de trabajo del proceso, que no tiene por qué coincidir con la carpeta del código fuente.

Método de Path	Uso
<code>toAbsolutePath()</code>	Obtiene una ruta absoluta.
<code>normalize()</code>	Elimina elementos redundantes como punto y resuelve el padre cuando es posible.
<code>getFileName()</code>	Devuelve el último elemento de la ruta.
<code>getParent()</code>	Devuelve el directorio padre.
<code>resolve(...)</code>	Construye una ruta hija.

```
Path directorio = Path.of("datos");
Path clientes = directorio.resolve("clientes.csv");
```

Evita construir rutas concatenando cadenas con separadores. `Path` utiliza el separador adecuado para cada sistema operativo.



¿Por qué no se pone la ruta como `datos/clientes.csv`? Porque tal vez es `datos\clientes.csv` ... Pero vaya, haz pruebas, ¡a ver qué pasa!

3. Comprobar el estado de una ruta

```
boolean existe = Files.exists(archivo);
boolean esArchivo = Files.isRegularFile(archivo);
boolean esDirectorio = Files.isDirectory(archivo);
```

```
boolean sePuedeLeer = Files.isReadable(archivo);
boolean sePuedeEscribir = Files.isWritable(archivo);
```

Estas comprobaciones sirven para informar al usuario, pero no garantizan que la operación posterior vaya a funcionar: el archivo podría desaparecer o cambiar de permisos entre ambas acciones. La operación debe manejar igualmente sus excepciones.

4. Crear archivos y directorios

```
Path directorio = Path.of("datos");
Files.createDirectories(directorio);

Path archivo = directorio.resolve("clientes.csv");
if (Files.notExists(archivo)) {
    Files.createFile(archivo);
}
```

- `createDirectory(ruta)` crea un único directorio y falla si no existe su padre.
- `createDirectories(ruta)` crea también los directorios intermedios que falten.
- `createFile(ruta)` crea un archivo vacío y falla si ya existe.

Normalmente no hace falta llamar a `createFile` antes de escribir: varios métodos de escritura crean el archivo automáticamente.

5. Leer un archivo de texto completo

readString

```
import java.nio.charset.StandardCharsets;

String contenido =
    Files.readString(archivo, StandardCharsets.UTF_8);
```

Devuelve todo el contenido como una sola cadena.

readAllLines

```
List<String> lineas =
    Files.readAllLines(archivo, StandardCharsets.UTF_
8);

for (String linea : lineas) {
    System.out.println(linea);
}
```

Carga todas las líneas en memoria y devuelve un `List<String>`.



Usa `readString` o `readAllLines` para archivos pequeños o medianos cuyo contenido completo necesites en memoria. No son apropiados para archivos potencialmente muy grandes.

6. Leer progresivamente

newBufferedReader: control explícito línea a línea

```
try (BufferedReader lector =
    Files.newBufferedReader(archivo, StandardCharsets.U
TF_8)) {

    String linea;
    while ((linea = lector.readLine()) != null) {
        procesar(linea);
    }
}
```

lines: procesamiento mediante Stream (programación funcional divertida)

```
try (Stream<String> lineas =
    Files.lines(archivo, StandardCharsets.UTF_8)) {

    lineas.filter(linea -> !linea.isBlank())
```

```
        .forEach(System.out::println);
    }
```

Tanto el lector como el `Stream` mantienen abierto un recurso del sistema. El bloque `try-with-resources` los cierra automáticamente, incluso si ocurre una excepción.

Necesidad	Opción
Todo el texto en una cadena	<code>readString</code>
Todas las líneas como lista	<code>readAllLines</code>
Lectura línea a línea con control explícito	<code>newBufferedReader</code>
Filtrar o transformar mediante Stream	<code>lines</code>

7. Escribir texto

writeString

```
String cabecera = "id;nombre;email"
    + System.lineSeparator();

Files.writeString(
    archivo,
    cabecera,
    StandardCharsets.UTF_8
);
```

Sin opciones adicionales, crea el archivo si no existe y reemplaza su contenido si ya existe.

write

```
List<String> lineas = List.of(
    "1;Ana;ana@example.com",
    "2;Luis;luis@example.com"
);

Files.write(archivo, lineas, StandardCharsets.UTF_8);
```

Permite escribir una colección de líneas e inserta entre ellas el separador de línea del sistema.

8. StandardOpenOption: decidir cómo se abre

Opciones habituales:

- `CREATE` : crea el archivo si no existe.
- `CREATE_NEW` : lo crea, pero falla si ya existe.
- `TRUNCATE_EXISTING` : vacía el contenido anterior.
- `APPEND` : añade al final.
- `WRITE` : abre para escritura.
- `READ` : abre para lectura.

```
import static java.nio.file.StandardOpenOption.APPEND;
import static java.nio.file.StandardOpenOption.CREATE;

Files.writeString(
    archivo,
    "3;Marta;marta@example.com" + System.lineSeparator
(),
    StandardCharsets.UTF_8,
    CREATE,
    APPEND
);
```



No uses `APPEND` para actualizar un registro dentro de un CSV. Para modificar o eliminar registros suele ser necesario leer los datos, transformarlos y reescribir el archivo completo.

9. Escritura progresiva con BufferedWriter

```
try (BufferedWriter escritor =
    Files.newBufferedWriter(archivo, StandardCharsets.U
TF_8)) {
```

```

    for (Cliente cliente : clientes) {
        escritor.write(cliente.toCsv());
        escritor.newLine();
    }
}

```

Es adecuada cuando se generan muchos registros uno a uno: evita construir previamente una cadena o lista enorme.

10. UTF-8 y codificación

La codificación determina cómo se convierten caracteres en bytes y viceversa. Conviene especificarla explícitamente mediante `StandardCharsets.UTF_8`.

Si se escribe con una codificación y se lee con otra, pueden dañarse tildes, eñes y símbolos. Para las prácticas del módulo, salvo indicación distinta, debe usarse UTF-8 tanto al leer como al escribir.

11. Archivos binarios

Las imágenes, PDF y otros archivos binarios no deben tratarse como texto.

```

byte[] datos = Files.readAllBytes(Path.of("foto.jpg"));
Files.write(Path.of("copia.jpg"), datos);

```

Para archivos grandes:

```

try (InputStream entrada = Files.newInputStream(origen);
     OutputStream salida = Files.newOutputStream(destino))
{

    entrada.transferTo(salida);

}

```

12. Copiar, mover y renombrar

```

import static java.nio.file.StandardCopyOption.REPLACE_EXISTING;

```

```
Files.copy(origen, destino, REPLACE_EXISTING);
Files.move(origen, destino, REPLACE_EXISTING);
```

Renombrar es mover a otra ruta:

```
Path actual = Path.of("datos", "clientes.tmp");
Path nuevo = Path.of("datos", "clientes.csv");

Files.move(actual, nuevo, REPLACE_EXISTING);
```

Puede solicitarse `ATOMIC_MOVE` cuando se necesita una sustitución indivisible, pero no todos los sistemas de archivos lo admiten.

13. Borrar

```
Files.delete(archivo);
boolean eliminado = Files.deleteIfExists(archivo);
```

`delete` falla si la ruta no existe. `deleteIfExists` devuelve `false` en ese caso. Para borrar un directorio, este debe estar vacío.

14. Listar y recorrer directorios

list: hijos directos

```
try (Stream<Path> elementos = Files.list(directorio)) {
    elementos.forEach(System.out::println);
}
```

walk: recorrido recursivo

```
try (Stream<Path> rutas = Files.walk(directorio)) {
    rutas.filter(Files::isRegularFile)
        .filter(ruta -> ruta.toString().endsWith(".csv"))
        .forEach(System.out::println);
}
```

find: recorrido con condición y profundidad máxima

```
try (Stream<Path> encontrados = Files.find(
    directorio,
    5,
    (ruta, atributos) ->
        atributos.isRegularFile()
        && ruta.toString().endsWith(".csv"))) {

    encontrados.forEach(System.out::println);
}
```

Los `Stream` obtenidos con `list`, `walk` y `find` también deben cerrarse.

15. Tamaño, fechas y atributos

```
long bytes = Files.size(archivo);
FileTime modificacion = Files.getLastModifiedTime(archivo);

BasicFileAttributes atributos =
    Files.readAttributes(archivo, BasicFileAttributes.class);

System.out.println(atributos.size());
System.out.println(atributos.creationTime());
System.out.println(atributos.lastModifiedTime());
System.out.println(atributos.isRegularFile());
```

Los atributos disponibles pueden variar según el sistema de archivos. No debe asumirse que Windows, Linux y macOS ofrecen exactamente los mismos.

16. Recursos temporales

```
Path temporal = Files.createTempFile("clientes-", ".tmp");
Path carpetaTemporal = Files.createTempDirectory("importacion-");
```

Java elige una ubicación temporal y un nombre sin colisión. Son útiles para preparar una escritura antes de sustituir el archivo definitivo.

17. Patrón seguro para reemplazar un archivo de datos

Escribir directamente sobre el archivo definitivo puede dejarlo incompleto si el proceso falla. Un patrón más robusto consiste en escribir primero un temporal y moverlo después.

```
import java.nio.file.AtomicMoveNotSupportedException;
import static java.nio.file.StandardCopyOption.ATOMIC_MOVE;
import static java.nio.file.StandardCopyOption.REPLACE_EXISTING;

Path destino = Path.of("datos", "clientes.csv");
Path padre = destino.toAbsolutePath().getParent();
Files.createDirectories(padre);

Path temporal =
    Files.createTempFile(padre, "clientes-", ".tmp");

try {
    Files.write(temporal, lineas, StandardCharsets.UTF_8);

    try {
        Files.move(temporal, destino,
            ATOMIC_MOVE, REPLACE_EXISTING);
    } catch (AtomicMoveNotSupportedException e) {
        Files.move(temporal, destino, REPLACE_EXISTING);
    }
} finally {
    Files.deleteIfExists(temporal);
}
```

Esto reduce el riesgo de destruir la versión válida anterior antes de terminar la nueva.

18. Excepciones habituales

Excepción	Significado habitual
<code>NoSuchFileException</code>	La ruta no existe.
<code>FileAlreadyExistsException</code>	Se intentó crear algo que ya existe.
<code>AccessDeniedException</code>	No hay permisos o el recurso está bloqueado.
<code>DirectoryNotEmptyException</code>	Se intentó borrar un directorio no vacío.
<code>AtomicMoveNotSupportedException</code>	No se admite el movimiento atómico.
<code>IOException</code>	Error general de entrada/salida.

```
try {
    List<String> lineas =
        Files.readAllLines(archivo, StandardCharsets.UTF_8);
} catch (NoSuchFileException e) {
    System.err.println("No existe: " + e.getFile());
} catch (AccessDeniedException e) {
    System.err.println("No hay permiso de lectura");
} catch (IOException e) {
    System.err.println("No se pudo leer: " + e.getMessage());
}
```

No conviene capturar una excepción para ignorarla. La aplicación debe recuperarse, informar o propagar el error a una capa capaz de decidir.

19. Errores frecuentes

- Confundir `Path.of(...)` con la creación física del archivo.
- Depender accidentalmente del directorio de trabajo.
- Omitir la codificación.
- Cargar un archivo enorme mediante `readAllLines`.
- Olvidar cerrar los recursos devueltos por `lines`, `list`, `walk` o `find`.
- Usar `APPEND` para actualizar o eliminar registros.
- Concatenar rutas como cadenas.

- Vaciar el archivo definitivo antes de generar correctamente los nuevos datos.
- Capturar `IOException` y continuar como si todo hubiese funcionado.
- Mezclar la interfaz de usuario con la lógica de persistencia.

20. Guía rápida de elección

Quiero...	Utilizar...
Leer un texto pequeño completo	<code>Files.readString</code>
Obtener todas sus líneas	<code>Files.readAllLines</code>
Procesar un archivo grande línea a línea	<code>Files.newBufferedReader</code> o <code>Files.lines</code>
Escribir una cadena	<code>Files.writeString</code>
Escribir una colección de líneas	<code>Files.write</code>
Generar muchas líneas progresivamente	<code>Files.newBufferedWriter</code>
Añadir contenido al final	<code>APPEND</code>
Copiar o mover	<code>Files.copy</code> o <code>Files.move</code>
Borrar sin fallar si no existe	<code>Files.deleteIfExists</code>
Listar un directorio	<code>Files.list</code>
Recorrer subdirectorios	<code>Files.walk</code> o <code>Files.find</code>

21. Documentación oficial

[Java Platform API — java.nio.file.Files \(Java 21\)](#)